Cyber Security Internship

Project Report

Task 1: Basic Network Sniffer

Task 2: Phishing Awareness Training

Intern name	OSEI DANIEL KWADWO
Intern ID	NO ID YET
Organization	CodeAlpha
Domain	Cyber Security
Date	20TH SEPTEMBER

GitHub repositories: CodeAlpha_BasicNetworkSniffer, CodeAlpha_PhishingAwarenessTraining

1. Summary

This report documents two projects completed for the CodeAlpha Cyber Security internship. The first is a Python program that captures and analyzes live network traffic. The second is an interactive training module that teaches people to recognize phishing attacks. Together they cover both sides of security: the technical (how data moves and how it can be observed) and the human (how attackers manipulate people).

	Task 1: Network Sniffer	Task 2: Phishing Training
Type	Python command-line tool	Interactive web module (HTML, CSS, JavaScript)
Main files	sniffer.py, README.md	phishing_awareness_training.html
Key skill	Packet capture, protocols, layers	Social engineering, awareness, user education
Tools	Python 3, Scapy	Web browser, no installation needed

2. Task 1: Basic Network Sniffer

2.1 Objective

Build a program that captures network packets, breaks them down into their layers, and displays useful details: source and destination addresses, protocol, ports, and payload.

2.2 Background concepts

- **Packet:** a small unit of data sent over a network. Every message (a web page, a DNS lookup, a ping) is split into packets.
- **Layers:** a packet is built like nested envelopes. Ethernet (local delivery), then IP (addresses), then TCP or UDP (ports, reliability), then the application data.
- **TCP vs UDP:** TCP is reliable and starts with a handshake (SYN, SYN-ACK, ACK). UDP is faster and has no handshake, used by DNS and streaming.
- **ICMP:** control messages, such as ping. **ARP:** maps an IP address to a hardware address on a local network. **DNS:** translates names like example.com into IP addresses.
- **Promiscuous capture:** the network card passes packets to the program, which is why admin privileges are required.

2.3 How the program works

Step	What happens	Code
1	Parse options: interface, packet count, filter, output file	argparse in main()
2	Start capture and hand every packet to a callback	sniff(prn=handle_packet)
3	Identify the network layer (ARP, IPv4, IPv6)	haslayer(IP), haslayer(ARP)
4	Identify the transport layer and read ports and flags	haslayer(TCP) / UDP / ICMP
5	Detect DNS queries and read the requested name	haslayer(DNSQR)
6	Show a payload preview as text or hex	payload_preview()
7	Count packets per protocol and print a summary on exit	collections.Counter

2.4 Key code explained

`sniff(iface=args.iface, filter=args.filter or None, prn=handle_packet, count=args.count, store=False)` **sniff()** is Scapy's capture function. **prn** is the function called for each packet. **filter** uses BPF syntax (for example *tcp port 80*) so the operating system discards unwanted packets early. **store=False** avoids keeping every packet in memory, so the sniffer can run for a long time.

```
if packet.haslayer(TCP):
tcp = packet[TCP]
    src, dst = f"{src}:{tcp.sport}", f"{dst}:{tcp.dport}"
extra = f"flags={tcp.sprintf('%flags%')}"
```

Scapy lets you index a packet by layer name. Once the TCP layer is found, ports and flags are read as simple attributes. The flags reveal the connection state: S is a connection request, SA is the reply, A is an acknowledgement, F ends the connection.

```
data = bytes(packet[Raw].load)[:limit] printable = sum(32 <= b <
127 or b in (9, 10, 13) for b in data) if printable / len(data)
> 0.8: return repr(text) return data.hex(' ')
```

The Raw layer is the application payload. If more than 80 percent of the bytes are printable characters it is shown as text, otherwise as hexadecimal. The preview is limited to 60 bytes to keep the output readable.

2.5 How to run

```
pip install scapy sudo python3 sniffer.py -c 50              # 50
packets sudo python3 sniffer.py -f "tcp port 80"      # HTTP only
sudo python3 sniffer.py -f icmp -o log.txt  # pings, save to file
```

On Windows, install Npcap and run the terminal as Administrator.

2.6 Sample output

```
[14:02:11] TCP   192.168.1.5:51234 -> 93.184.216.34:80  len=74  flags=S [14:02:11]
DNS   192.168.1.5:55120 -> 8.8.8.8:53  len=72  query=example.com.
[14:02:12] ICMP  192.168.1.5 -> 8.8.8.8  len=84  type=8
--- Capture summary ---
TCP     31
DNS     12
ICMP     7
```

2.7 What I learned

- Data crosses the network as layered packets, and each layer has a clear job.
- Unencrypted protocols such as HTTP, FTP and Telnet expose their contents. HTTPS payloads appear as unreadable bytes, which shows why encryption matters.
- Packet analysis is the foundation of intrusion detection, network troubleshooting and digital forensics.
- Monitoring traffic has legal and ethical limits: it is only acceptable on networks I own or have permission to monitor.

2.8 Limitations and future work

- It prints to the screen and does not save a .pcap file. Scapy's wrpcap() could add this.
- It does not reassemble TCP streams or decode application protocols such as HTTP fully.
- A detection feature could flag port scans (many SYN packets from one address) and d
- isplay a live chart.

3. Task 2: Phishing Awareness Training

3.1 Objective

Create an engaging module that explains how to recognize phishing emails and fake websites, describes the social engineering tactics attackers use, gives best practices, includes real-world examples, and tests the learner with an interactive quiz.

3.2 Why this matters

Technical defences can be bypassed if a person is tricked into handing over a password or running a file. Phishing is one of the most common ways attackers gain their first access to an organization, so training people is a security control in its own right.

3.3 Structure of the module

Section	Content	Purpose
Would you have clicked?	A fake email with 5 clickable red flags	Learning by doing, and a memorable opening
Recognize phishing emails	Sender, urgency, links, greeting, attachments, unusual requests	Checklist for daily use
Social engineering tactics	Urgency, authority, greed, familiarity; smishing, vishing, quishing; business email compromise	Explains why phishing works
Fake websites	Look-alike domains, subdomain tricks, padlock myth	Teaches how to read a web address
Real-world cases	RSA 2011, Google and Facebook invoice fraud, Twitter 2020	Shows real consequences
Best practices	Do and do-not lists, what to do if you clicked	Actionable guidance
Quiz	6 scenario questions with instant feedback and a score	Reinforces learning and measures it

3.4 The red flags in the sample email

Red flag	Explanation
Sender: security-alert@micros0ft-support.co	Look-alike domain: a zero replaces the letter o, and the domain is not Microsoft's.
Subject: account closed in 24 hours	Artificial urgency, intended to stop careful thinking.
Greeting: Dear Customer	Generic greeting. Genuine providers normally use your name.
Button links to login-secure.xyz	The visible text and the real destination do not match.
Attachment: Account_Form.html.exe	A double extension hides an executable file. Opening it can install malware.

3.5 How the web page works

- **HTML/CSS:** a single self-contained page that works on phones and desktops and supports light and dark modes.

- **Red-flag explorer (JavaScript):** each suspicious detail is a button. Clicking it shows the explanation and updates a counter (0 of 5). The JavaScript keeps a record of which flags were found.
- **Quiz engine (JavaScript):** questions are stored in an array with the options, the correct answer index and an explanation. After each answer the correct option is highlighted, feedback is shown, and the score is totalled at the end, with a retry button.
- **Accessibility:** keyboard focus is visible, controls are real buttons, and feedback is announced to screen readers.

3.6 How to use it in training

Open the page, let participants try the email exercise as a group, discuss the red flags, then have each person complete the quiz. A score of 5 or 6 out of 6 shows good understanding. Lower scores indicate the sections to revisit.

3.7 What I learned

- Attackers exploit emotion (fear, urgency, curiosity, trust) more than software flaws.
- A padlock and HTTPS do not prove a site is legitimate. Reading the domain correctly is the key skill.
- Phishing is not only email: SMS, phone calls and QR codes are also used.
- Reporting quickly after a mistake limits the damage, so training should encourage reporting and not blame.

4. Skills gained and why they matter

Skill	Evidence in this internship	Where it is used in a career
Network protocol analysis	Task 1: decoding Ethernet, IP, TCP, UDP, ICMP, DNS, ARP	SOC analyst, network engineer, incident response
Python for security	Task 1: Scapy, argparse, callbacks, data handling	Security automation, tooling, scripting
Threat awareness	Task 2: phishing indicators and social engineering	Security awareness, blue team, risk management
Security communication	Task 2: explaining risk in plain language with examples	Training, consulting, reporting to non-technical staff
Ethics and responsibility	Both: permission-based monitoring, honest reporting	Professional conduct, legal compliance
Documentation and delivery	GitHub repos, README, report, video, LinkedIn post	Teamwork, portfolio, professional presence

How these tasks connect

A sniffer shows what is actually travelling over a network, including the DNS lookups and connections that a phishing click would trigger. Phishing awareness reduces the chance that such a click happens at all. One is detection and visibility, the other is prevention, and a security team needs both.

5. Conclusion

These projects gave practical experience of how networks carry data and how people can be manipulated through that same network. The sniffer builds technical understanding and Python skills, and the phishing module builds the ability to explain security to others. Both reinforce that security depends on technology, people and responsible behaviour working together.